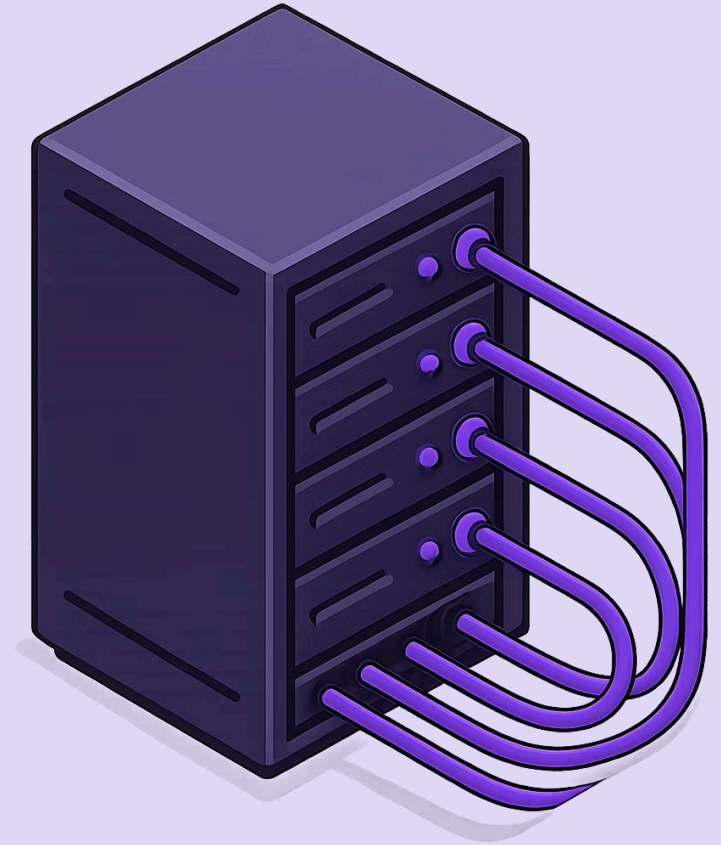


OpenStack Monitoring & Troubleshooting

Kolla-Ansible Deployment — Module Guide for System Administrators

By the end of this module, you will be able to identify failing services, read Nova and Neutron logs, restart containers safely, debug failed VM launches, and verify complete cluster health across a production Kolla-Ansible OpenStack deployment.



Module Objectives

What You Will Learn



Understand Logs

Navigate the `/var/log/kolla/` directory structure and interpret service-specific log files.



Identify Failures

Recognize symptoms of failing services across Nova, Neutron, Keystone, Cinder, and more.



Container Health

Use `docker ps -a` to check container states and investigate unhealthy or exited containers.



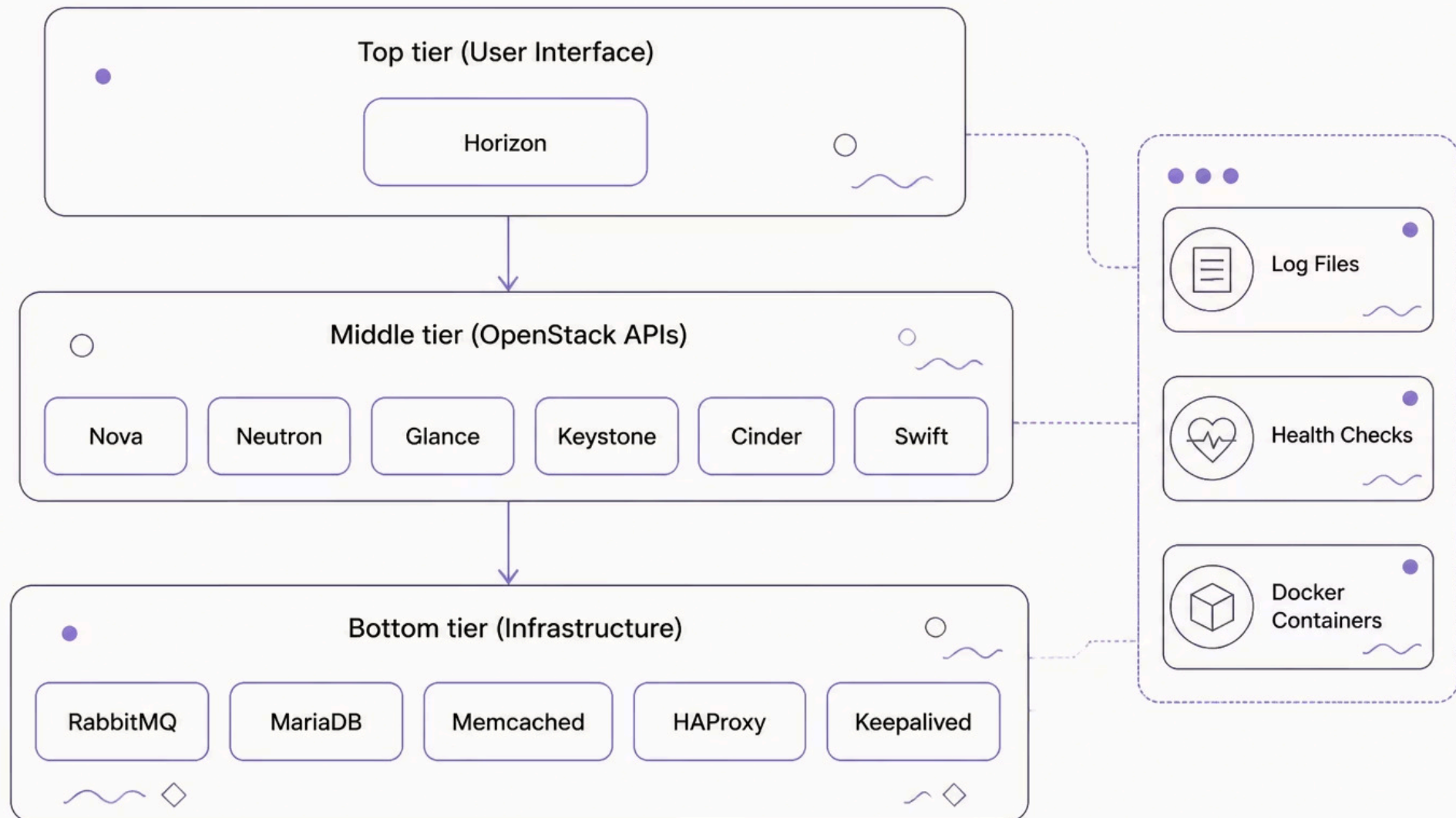
Debug & Recover

Follow a structured debugging workflow to fix failed VM launches and restore cluster health.

- ❏ Kolla-Ansible stores service logs under `/var/log/kolla/` on the host, mapped from containers. Always run `kolla-ansible prechecks` before deployments.

OpenStack Monitoring Architecture

A Kolla-Ansible deployment spans multiple tiers — from the user-facing Horizon dashboard through core OpenStack APIs, down to the messaging, database, and load-balancing infrastructure. Understanding each layer is essential before diving into any troubleshooting effort.



Where Are Logs Stored?

In a Kolla-Ansible deployment, every service writes its logs to a dedicated subdirectory under `/var/log/kolla/` on the host. These logs are **persisted on the host filesystem** even though the services themselves run inside Docker containers — making them accessible without entering containers.

Log Root Directory

`/var/log/kolla/`

- `nova/`
- `neutron/`
- `keystone/`
- `glance/`
- `cinder/` · `swift/`
- `rabbitmq/` · `mariadb/`
- `haproxy/` · `horizon/`

Key Log Files

Nova:

- `nova-api.log`
- `nova-compute.log`
- `nova-scheduler.log`
- `nova-conductor.log`

Neutron:

- `neutron-server.log`
- `openvswitch-agent.log`
- `l3-agent.log` · `dhcp-agent.log`
- `metadata-agent.log`



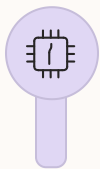
Monitoring Layers — Always Investigate Layer by Layer

Never jump directly to Nova logs when a VM fails. Every issue must be investigated systematically from the top of the stack down. Skipping layers leads to missed root causes and wasted time. Follow this seven-layer model on every incident.



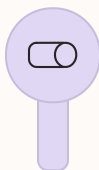
Layer 7 — Virtual Machine

Guest OS, cloud-init, and application-level errors inside the VM itself.



Layer 6 — Hypervisor

libvirt / KVM errors, compute node resources, and hardware health.



Layer 5 — Open vSwitch

OVS bridge configuration, port mappings, and flow table correctness.



Layer 4 — MariaDB

Database connectivity, replication health, and query failures.



Layer 3 — RabbitMQ

Message queue health, consumer count, and inter-service communication.



Layer 2 — OpenStack Services

Nova, Neutron, Keystone, Glance, Cinder service status and logs.



Layer 1 — Docker Containers

Start here: verify all containers are Up and Healthy before anything else.

Operational Commands

Service Health Checks



Check Docker Containers

Run `docker ps -a` to list all containers. Expected state is **Up (healthy)**.

Investigate immediately if you see:

- `Exited` — container crashed or failed to start
- `Restarting` — container is in a restart loop
- `Unhealthy` — health check is failing

Follow up with `docker logs <container_name>` for details.



Check OpenStack Services

Use the OpenStack CLI to verify service state:

- `openstack compute service list`
- `openstack network agent list`
- `openstack volume service list`

Every service should show **State = Up** and **Status = Enabled**. Also verify hypervisors and resource availability:

- `openstack hypervisor list` — check Up/Enabled
- `openstack hypervisor stats show` — verify RAM, vCPUs, Disk

Quick Reference

Common Service Failures by Symptom

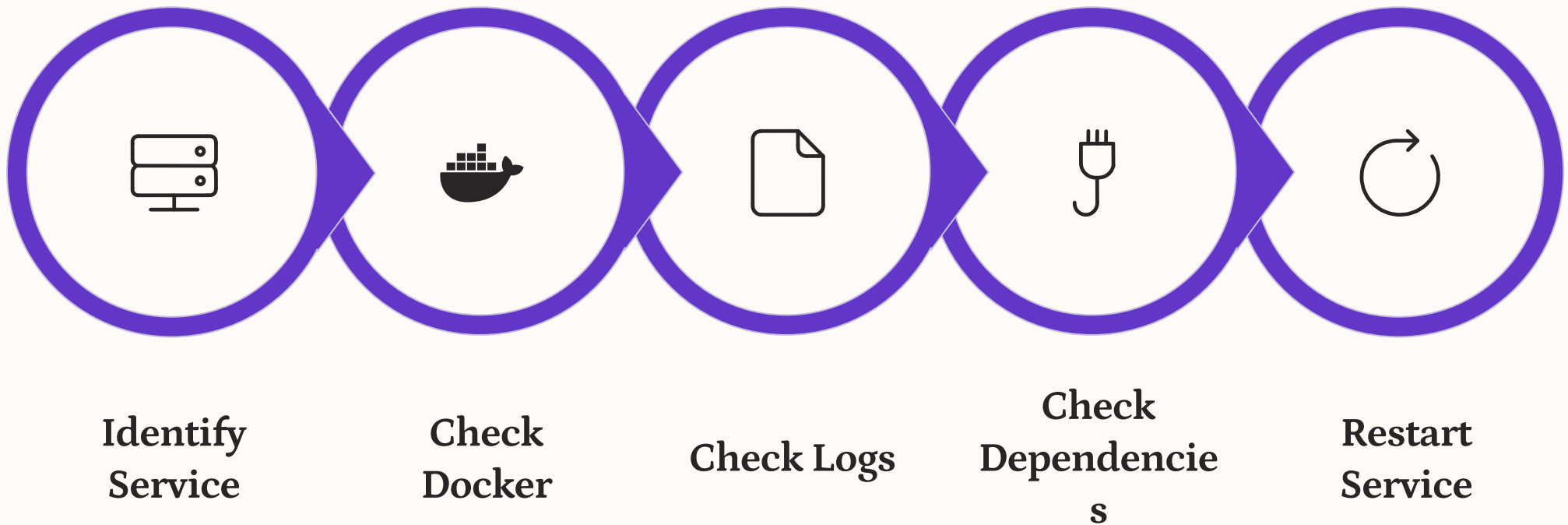
Use this table to quickly map user-reported symptoms to the likely failing component. Always confirm with logs before taking corrective action.

Symptom	Likely Cause	Where to Check
Cannot login to Horizon	Keystone failure	keystone/ logs
VM stuck in Build state	Nova Compute / Scheduler	nova-compute.log
No IP assigned to VM	Neutron DHCP Agent	dhcp-agent.log
Floating IP unreachable	Router / L3 Agent	l3-agent.log
Volume attach failed	Cinder + iSCSI	cinder/ logs
Image download failed	Glance	glance/ logs
Scheduling failed	Nova Scheduler / Placement	nova-scheduler.log
Container restart loop	RabbitMQ / MariaDB	Docker + service logs

Core Workflow

Debugging Methodology

Always follow this structured workflow. Resist the urge to jump to a fix before fully diagnosing the root cause. This approach mirrors how production incidents are handled in enterprise environments.



Example: A failed VM launch starts with Nova, then checks Nova logs, then RabbitMQ messaging, Neutron networking, and Placement API — resolving each dependency before retesting the launch.



Labs 1 & 2

Inspecting Nova & Neutron Logs

LAB 1 — Nova Logs

Locate the Nova log directory and open the API and Compute logs. Observe instance creation entries as a VM is launched, then search for **ERROR** and **WARNING** entries.

Expected learning:

- API receives the launch request
- Scheduler selects a compute node
- Compute node creates the VM

LAB 2 — Neutron Logs

Create a Network → Subnet → Router, then inspect logs. Observe network creation, port creation, DHCP allocation, and router interface attachment.

Then: intentionally create an **invalid network** and compare the resulting error entries against the successful creation logs to understand what failure looks like.

Labs 3 & 4

Restarting Services & Troubleshooting Failed VM Launches



LAB 3 — Restart Services

Restart services one at a time: Horizon, Nova API, Neutron Server, Glance API. After each restart, verify the container returns to **Healthy** and the service responds correctly via the OpenStack CLI. **Never restart all services simultaneously** — isolate each restart to understand its impact.



LAB 4 — Failed VM Launch (Enterprise Lab)

Launch a VM, confirm it reaches **ACTIVE**. The instructor then intentionally disables Nova Compute, a Neutron Agent, or RabbitMQ. Students must identify which service failed, inspect container state and logs, fix the issue, and relaunch the VM. This directly simulates a real production incident response.

⚠ Do not restart all containers simultaneously — this can cascade failures across dependent services and is harder to diagnose.

LAB 5

Verify Complete Cluster Health

Before declaring a cluster production-ready, systematically verify every tier. This checklist mirrors the final verification step of a real deployment runbook.



Controller

- All containers healthy
- API services responding
- Keystone available
- Glance available



Compute

- Hypervisor registered
- Nova Compute Up
- OVS bridges present
- Neutron agents Up



Storage

- Cinder Volume Up
- iSCSI target running



Messaging & DB

- RabbitMQ running
- MariaDB healthy

Final Troubleshooting Checklist

Use this ordered checklist for any production incident. Work top to bottom — do not skip steps. A cluster is only healthy when every item below passes.

01

Horizon Login

Confirm Keystone and Horizon containers are healthy and users can authenticate.

02

Docker Containers

All containers show Up (healthy). No Exited or Restarting states.

03

OpenStack Services

All compute, network, and volume services show State=Up, Status=Enabled.

04

RabbitMQ & MariaDB

Messaging and database layers are healthy — critical for all service coordination.

05

Nova Compute & Neutron

Nova Compute Up, Neutron agents Up, OVS bridges correctly configured.

06

VM Lifecycle

VM can launch → receive IP → reach Internet → attach Volume.

07

Logs Clean

No unresolved ERRORS in any service log. Cluster is healthy. 

Enterprise Troubleshooting Flow

When a user reports a failed VM, follow this escalation path:



Check Horizon Error

Note the exact error message displayed in the dashboard.



Nova API → Scheduler → Compute

Trace the request through Nova's three core log files.



Placement → Neutron → OVS

Verify resource availability and network plumbing.



RabbitMQ → MariaDB → Hypervisor

Confirm messaging, database, and compute layer are all functional.



A clean run through this checklist confirms your Kolla-Ansible cluster is production-ready.